DALHOUSIE UNIVERSITY

Tree-Based Models

Recursive partitioning, regression trees, and pruning

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 The running example

What it is for Every idea in these notes is shown on one data set, small enough to check by hand and drawn so that the right answer is known.

The data 60 cases with two numeric predictors, x_1 and x_2, and a numeric target y. The plane is cut into the five regions of Figure 1, each with its own mean, and every case's y is its region's mean plus normal noise with standard deviation 0.6. Table 1 counts how many of the 60 cases fall in each region. A good tree for these data has five leaves, one per region, and a leaf that mixes two regions, or splits one, is a mistake we can see.

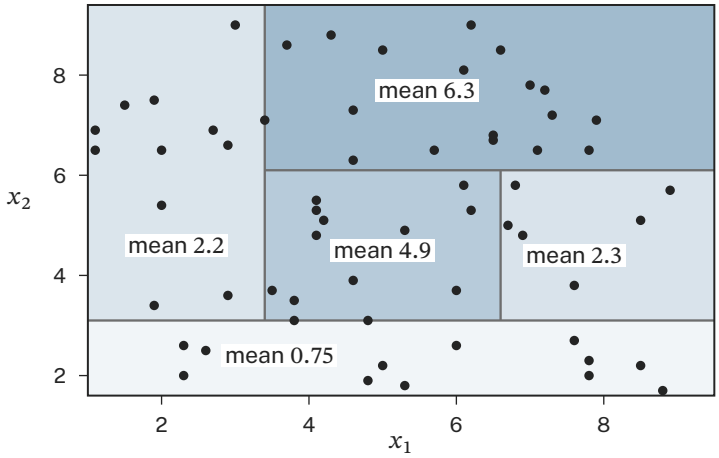


Figure 1. The five regions the 60 cases are drawn from, each labelled with its mean, and the 60 cases as points

Table 1. The five regions, their means and how many of the 60 cases fall in each

Region	mean	cases
$x_2 < 3.1$	0.75	12
$x_2 \geq 3.1,\ x_1 < 3.4$	2.2	11
$3.4 \leq x_1 < 6.6,\ 3.1 \leq x_2 < 6.1$	4.9	13
$x_1 \geq 6.6,\ 3.1 \leq x_2 < 6.1$	2.3	6
$x_1 \geq 3.4,\ x_2 \geq 6.1$	6.3	18

The Python listing draws the cases with seed 23. R's random numbers differ from NumPy's, so the R listing types the same 60 rows in, and the two languages fit the same data.

R

```r
d <- data.frame(     # the 60 cases
  x1 = c(6.6, 6.1, 2.0, 1.9, 6.2, 7.8, 2.6, 2.7,
    6.7, 4.8, 4.3, 3.8, 1.5, 4.6, 3.4, 4.1,
    5.3, 6.5, 6.0, 6.9, 1.1, 6.2, 5.3, 7.8,
    8.5, 1.1, 7.6, 3.0, 6.0, 7.1, 7.8, 8.5,
    6.1, 7.9, 5.0, 2.9, 2.3, 5.0, 7.0, 4.6,
    6.5, 2.0, 8.8, 4.8, 7.6, 6.8, 7.2, 4.1,
    1.9, 4.6, 3.7, 4.2, 8.9, 3.5, 7.3, 2.3,
    5.7, 3.8, 2.9, 4.1),
  x2 = c(8.5, 5.8, 5.4, 7.5, 5.3, 2.3, 2.5, 6.9,
    5.0, 3.1, 8.8, 3.5, 7.4, 3.9, 7.1, 5.5,
    1.8, 6.8, 3.7, 4.8, 6.9, 9.0, 4.9, 6.5,
    5.1, 6.5, 2.7, 9.0, 2.6, 6.5, 2.0, 2.2,
    8.1, 7.1, 2.2, 3.6, 2.0, 8.5, 7.8, 7.3,
    6.7, 6.5, 1.7, 1.9, 3.8, 5.8, 7.7, 4.8,
    3.4, 6.3, 8.6, 5.1, 5.7, 3.7, 7.2, 2.6,
    6.5, 3.1, 6.6, 5.3),
  y = c(5.43, 3.92, 1.97, 3.35, 5.01, 1.91,
    1.02, 2.47, 1.95, 4.89, 7.37, 5.61,
    1.82, 4.62, 7.1, 4.59, 1.3, 5.75, 5.28,
    3.22, 2.7, 6.45, 4.98, 6.25, 2.29, 2.91,
    1.44, 2.44, -0.01, 5.99, 0.93, 0.78,
    5.49, 6.7, 2.31, 1.46, 0.79, 7.27, 6.05,
    6.06, 6.4, 2.54, 0.5, 1.34, 1.16, 2.69,
    6.52, 4.8, 2.43, 6.33, 7.1, 4.62, 2.28,
    4.07, 5.63, 0.61, 5.37, 4.39, 1.4, 4.6))
```

Python

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score
from sklearn.model_selection import StratifiedKFold as SKF
from sklearn.tree import DecisionTreeClassifier as DTC
from sklearn.tree import DecisionTreeRegressor as DTR
from sklearn.tree import export_text
def region(x1, x2):  # the mean of each of five regions
    if x2 < 3.1:
        return 0.75
    if x1 < 3.4:
        return 2.2
    if x2 >= 6.1:
        return 6.3
    if x1 < 6.6:
        return 4.9
    return 2.3

def draw(n, seed):   # n cases, normal noise with sd 0.6
    rng = np.random.default_rng(seed)
    x1 = np.round(rng.uniform(1, 9, n), 1)
    x2 = np.round(rng.uniform(1.5, 9, n), 1)
    mu = np.array([region(a, b)
                   for a, b in zip(x1, x2, strict=True)])
    y = np.round(mu + rng.normal(0, 0.6, n), 2)
    return pd.DataFrame({"x1": x1, "x2": x2, "y": y})

d = draw(60, 23)                    # the 60 cases
X, y = d[["x1", "x2"]], d["y"]
```

2 Regression trees

What it is for A classification tree predicts a class. A **regression tree** predicts a number, here y. The recursive partitioning algorithm is unchanged. At each node it lists every question each predictor allows, scores each one, keeps the best, splits, and repeats inside each side. Only two things are swapped, what a leaf predicts and how "mixed" a box is. Table 2 sets the two kinds of tree side by side.

Table 2. Classification versus regression trees by target type, leaf prediction and split criterion

	Classification tree	Regression tree
Target	nominal	numeric
A leaf predicts	the majority class of its cases (and the class distribution)	the mean of its cases' target values
A split is scored by	impurity reduction: Gini or information gain	error reduction: the drop in within-node sum of squares
In `rpart`	`method = "class"`	`method = "anova"`
In scikit-learn	`DecisionTreeClassifier`	`DecisionTreeRegressor`

2.1 What a leaf predicts, and why the mean

The rule A leaf must predict one number k for every case that reaches it. Score a candidate k by the **mean squared error** of the leaf's cases,

$$\mathrm{Err} = \frac{1}{n}\sum_i (y_i - k)^2,$$

the average squared distance from each case to the prediction. The number that makes this smallest is the mean of the leaf's targets, so a leaf predicts its mean. The mean is the balance point of the cases, and moving k away from it in either direction adds more squared error on the far side than it removes on the near side.

Worked example One leaf of the tree grown below holds six cases, with targets 1.95, 3.22, 2.29, 1.16, 2.69 and 2.28. Their mean is 2.265. Guessing 2 gives mean squared error 0.473, guessing 2.5 gives 0.458, and the mean gives 0.402, the smallest of the three and of any guess. At the mean the six squared distances are 0.099, 0.912, 0.001, 1.221, 0.181 and 0.000. Their sum, 2.414, is the leaf's **sum of squared errors** (SSE), and dividing by the 6 cases gives $\text{Err} = 2.414/6 = 0.402$. Err is the leaf's variance, computed with n rather than $n-1$ in the denominator, and SSE is $n \times \text{Err}$. Err plays the part Gini played for a class. It is 0 when every case in the leaf has the same target.

R

```r
leaf <- with(d, y[x1 >= 6.45 & x2 >= 2.9 &
                  x2 < 6.05])      # one leaf, 6 cases
leaf  # 1.95 3.22 2.29 1.16 2.69 2.28
round(sapply(c(2, 2.5, mean(leaf)),
             function(k) mean((leaf - k)^2)), 3)
# 0.473 0.458 0.402, the mean 2.265 is lowest
dev2 <- (leaf - mean(leaf))^2       # squared distances
round(dev2, 3); round(sum(dev2), 3)
# 0.099 0.912 0.001 1.221 0.181 0.000, and 2.414
```

Python

```python
leaf = y[(X["x1"] >= 6.45) & (X["x2"] >= 2.9)
         & (X["x2"] < 6.05)]        # one leaf, 6 cases
leaf.tolist()  # [1.95, 3.22, 2.29, 1.16, 2.69, 2.28]
[round(np.mean((leaf - k) ** 2), 3)
 for k in (2, 2.5, leaf.mean())]
# [0.473, 0.458, 0.402], the mean 2.265 is lowest
dev2 = (leaf - leaf.mean()) ** 2    # squared distances
dev2.round(3).tolist(), round(dev2.sum(), 3)
# [0.099, 0.912, 0.001, 1.221, 0.181, 0.0], 2.414
```

Reading it in the output The libraries keep these two numbers for every node, but not the same one. R's printout of a fitted tree gives one line per node, and this leaf's line is `28) x1>=6.45 6 2.41 2.26 *`, the node number, the answer that leads to it, its 6 cases, its SSE (2.41), its mean (2.26) and a star for a leaf (Section 2.3 prints the whole tree). scikit-learn keeps Err rather than SSE, in `tree_.impurity`, alongside the sizes in `tree_.n_node_samples` and the means in `tree_.value`. For the same leaf it holds 6 cases and 0.402.

2.2 How a question is scored

The rule A question T splits the n cases of a node into the n_T that answer yes and the $n_{\neg T}$ that answer no. The **pooled error** after the split is the weighted average of the two sides' errors,

$$\text{Err}_T = \frac{n_T}{n}\,\text{Err}(\text{yes side}) + \frac{n_{\neg T}}{n}\,\text{Err}(\text{no side}),$$

and the question is worth the **error reduction** $\text{Err} - \text{Err}_T$. As for Gini, each side counts for the share of cases it holds, so a tiny side with a small error cannot win on its own. The best question for a node is the one with the largest reduction over every predictor.

Worked example All 60 cases have $\text{Err} = 4.4986$. The question $x_2 < 2.9$ sends 12 cases one way, with mean 1.08 and $\text{Err} = 0.3661$, and 48 the other, with mean 4.45 and $\text{Err} = 3.2525$ (Figure 2). The pooled error is

$$\tfrac{12}{60}(0.3661) + \tfrac{48}{60}(3.2525) = 2.6752,$$

so the question removes $4.4986 - 2.6752 = 1.823$. The search tries a cut halfway between each pair of neighbouring values, 42 cuts on x_2 and 38 on x_1. The best cut on x_1, $x_1 < 3.2$, removes only 0.968, so $x_2 < 2.9$ becomes the root. It separates the low region, mean 0.75, from everything above it, which is the largest single drop in error the plane offers.

The same drop in sums of squares Multiplying each side's Err by its size turns the pooled error into a plain sum, so the reduction can also be read as the parent's SSE minus the SSE of its two sides, with no weights at all. Here $60 \times 4.4986 = 269.915$, $12 \times 0.3661 = 4.393$ and $48 \times 3.2525 = 156.118$, and

$$269.915 - 4.393 - 156.118 = 109.404 = 60 \times 1.8234.$$

Both forms pick the same question, since one is the other times n. Pruning, below, prices questions in this second form.

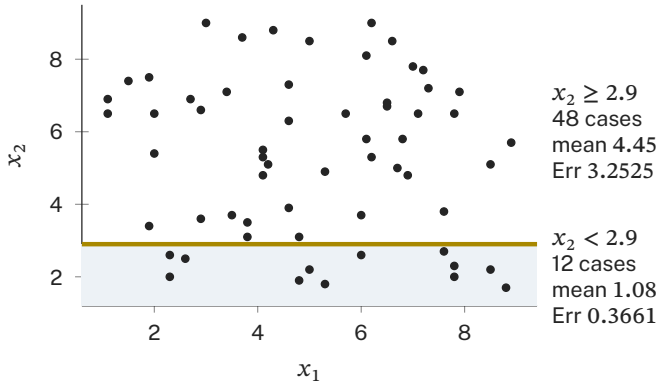


Figure 2. The root question $x_2 < 2.9$ on the 60 cases, with each side's size, mean and error

R

```r
err <- function(v) mean((v - mean(v))^2) # Err(D)
gains <- function(x, y) {   # error removed per cut
  v <- sort(unique(x))
  cuts <- (head(v, -1) + tail(v, -1)) / 2
  g <- sapply(cuts, function(c) {
    l <- y[x < c]; r <- y[x >= c]
    err(y) - (length(l) * err(l) +
              length(r) * err(r)) / length(y)
  })
  list(cuts = cuts, g = g)
}
round(err(d$y), 4)                  # 4.4986
for (col in c("x1", "x2")) {  # best cut per predictor
  s <- gains(d[[col]], d$y)
  cat(col, length(s$cuts), s$cuts[which.max(s$g)],
      round(max(s$g), 4), "\n")
}
# x1 38 3.2 0.9684
# x2 42 2.9 1.8234
low <- d$x2 < 2.9                   # the root question
sapply(split(d$y, low), function(s) round(c(
  n = length(s), mean = mean(s), err = err(s)), 4))
#        FALSE    TRUE
# n    48.0000 12.0000
# mean  4.4525  1.0767
# err   3.2525  0.3661
pooled <- mean(low) * err(d$y[low]) +
  mean(!low) * err(d$y[!low])       # weighted Err
round(pooled, 4)                    # 2.6752
sums <- sapply(list(d$y, d$y[low], d$y[!low]),
  function(s) length(s) * err(s))   # n x Err = SSE
round(sums, 3); round(sums[1] - sums[2] - sums[3], 3)
# 269.915 4.393 156.118, and 109.404 = 60 x 1.8234
```

Python

```python
def err(v):                         # Err(D)
    return np.mean((v - v.mean()) ** 2)

def gains(x, y):          # error removed by each cut
    v = np.unique(x)
    cuts = (v[:-1] + v[1:]) / 2     # midpoints
    g = [err(y) - (len(y[x < c]) * err(y[x < c])
         + len(y[x >= c]) * err(y[x >= c])) / len(y)
         for c in cuts]
    return cuts, np.array(g)
round(err(y), 4)                    # 4.4986
for col in ["x1", "x2"]:   # best cut per predictor
    cuts, g = gains(X[col].to_numpy(), y.to_numpy())
    print(col, len(cuts), round(cuts[g.argmax()], 2),
          round(g.max(), 4))
# x1 38 3.2 0.9684
# x2 42 2.9 1.8234
low = X["x2"] < 2.9                 # the root question
[(len(s), round(s.mean(), 2), round(err(s), 4))
 for s in (y[low], y[~low])]
# [(12, 1.08, 0.3661), (48, 4.45, 3.2525)]
pooled = (low.mean() * err(y[low])
          + (~low).mean() * err(y[~low]))
round(pooled, 4)                    # 2.6752
sums = [len(s) * err(s) for s in (y, y[low], y[~low])]
drop = sums[0] - sums[1] - sums[2]  # parent - sides
np.round(sums, 3), round(drop, 3)
# [269.915 4.393 156.118] 109.404, or 60 x 1.8234
```

`gains` is the search of the classification tree with `err` in place of Gini. Each printed line of the loop is a predictor, the number of cuts tried, the best cut, and the error that cut removes.

2.3 The grown tree

The search then runs again inside each side, on that side's cases alone, and again inside each of their sides. Growth stops only where every split would leave a leaf with fewer than 6 cases. The result has nine leaves (Figure 3), and its training SSE, the sum of the nine leaves' SSE, is 15.95, down from 269.92 for the root alone.

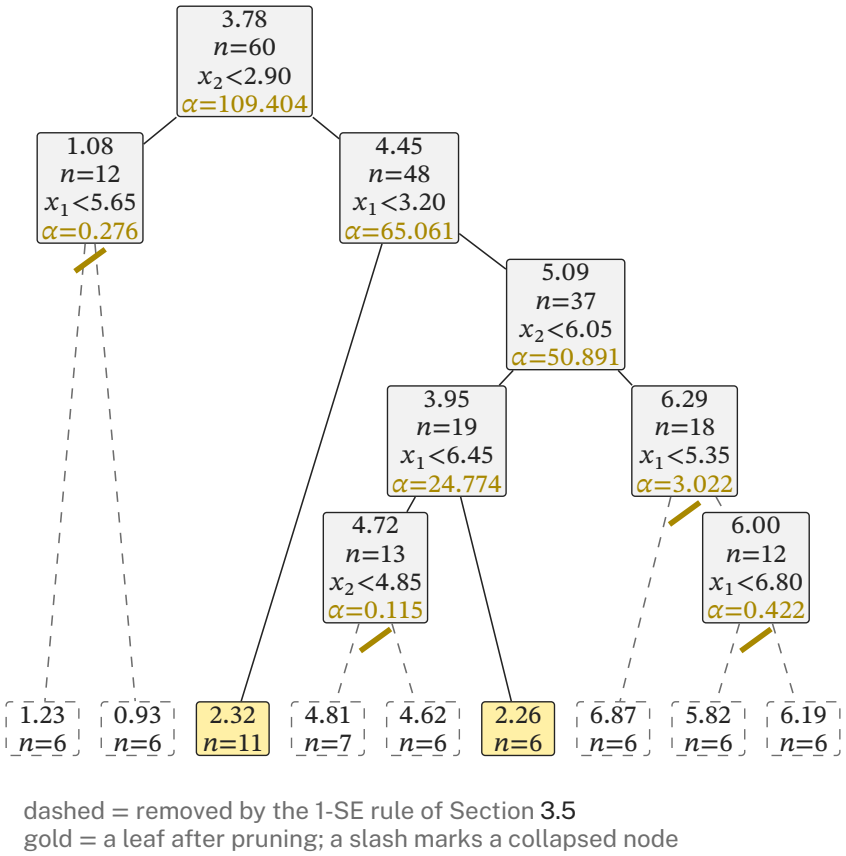


Figure 3. The grown nine-leaf tree. Each node shows its mean, its size and, if it is internal, its question and the price α at which pruning removes it (Section 3.3). The four dashed branches are the ones pruning removes.

R prints the same tree one node per line. Indentation is depth, the children of node i are nodes $2i$ and $2i + 1$, and each line gives the answer that leads to the node, its size, its SSE (R calls it `deviance`) and its mean (`yval`). The root's line reads 60 cases, SSE 270, mean 3.78, and node 28 is the leaf of the worked example.

R

```
print(fit, digits = 3)  # one line per node
#  1) root 60 270.000 3.780
#    2) x2< 2.9 12   4.390 1.080
#      4) x1>=5.65 6   2.310 0.925 *
#      5) x1< 5.65 6   1.810 1.230 *
#    3) x2>=2.9 48 156.000 4.450
#      6) x1< 3.2 11   3.610 2.320 *
#      7) x1>=3.2 37  87.400 5.090
#       14) x2< 6.05 19  29.700 3.950
#         28) x1>=6.45 6   2.410 2.260 *
#         29) x1< 6.45 13   2.520 4.720
#           58) x2>=4.85 6   0.773 4.620 *
#           59) x2< 4.85 7   1.630 4.810 *
#       15) x2>=6.05 18   6.850 6.290
#         30) x1>=5.35 12   2.370 6.000
#           60) x1< 6.8 6   1.200 5.820 *
#           61) x1>=6.8 6   0.746 6.190 *
#         31) x1< 5.35 6   1.460 6.870 *
```

Nine leaves for five true regions means four splits divide cases that come from one region. Each of those splits lowered the training error, because any split does, but the drop came from the noise in these particular 60 targets. The rest of these notes is about finding and removing them.

2.4 Predicting with a regression tree

A new case is sent down the tree like a day in the weather tree, and the prediction is the mean of the leaf it reaches. The case $(x_1, x_2) = (5, 7)$ lands in a leaf of six cases with mean 6.87. The prediction is therefore constant inside each leaf's rectangle and jumps at each cut, a step function rather than a slope. It also can never leave the range of the training targets. The case $(9.5, 9.5)$ lies outside every training point, and the tree predicts 6.19, a leaf mean, where linear regression extrapolates to 7.33. The largest training y is 7.37, and no tree fitted to these data can predict above it.

R

```
new <- data.frame(x1 = c(5, 9.5), x2 = c(7, 9.5))
round(predict(fit, new), 2)     # 6.87 6.19, 9 leaves
round(predict(pruned, new), 2)  # 6.29 6.29, 5 leaves
```

Python

```
new = pd.DataFrame({"x1": [5, 9.5], "x2": [7, 9.5]})
base.predict(new).round(2)    # [6.87 6.19], 9 leaves
pruned.predict(new).round(2)  # [6.29 6.29], 5 leaves
```

3 Pruning

What it is for Training error cannot say when to stop growing, since it always prefers another split. Grown with no stopping rule at all, the tree puts one case in each leaf, 60 leaves with training SSE 0. A minimum leaf size of 6 stops at nine leaves, and the drops its eight splits buy show what a rule fixed in advance cannot see. Four of them remove 109.4, 65.1, 50.9 and 24.8 in SSE, and four remove 3.02, 0.42, 0.28 and 0.12, all inside one region. A rule that only looks at leaf sizes treats them alike. Make the rule stricter and it loses a real region instead. `rpart`'s default smallest leaf is 7 cases, and the region with mean 2.3 holds only 6, so its split is never made and the default tree has four leaves (Table 6 lists the defaults).

R

```
small <- rpart(y ~ x1 + x2, data = d)   # defaults
sum(small$frame$var == "<leaf>")    # 4 leaves
rpart.control()$minbucket           # 7, above 6 cases
```

Python

```
big = DTR(random_state=3141).fit(X, y)
big.get_n_leaves(), big.get_depth()  # 60 leaves, depth 12
((big.predict(X) - y) ** 2).sum()   # 0.0, one case a leaf
```

Stopping early in this way is called **pre-pruning**. It has a second blind spot. A rule that refuses a split because the split removes little never sees the splits beneath it. Four cases at the corners of a square, with y equal to 0, 10, 10 and 0 in a checkerboard, show it. Any first question leaves a 0 and a 10 on each side, so every cut on x_1 or x_2 removes nothing, yet one more question on each side makes every leaf exact. **Post-pruning** avoids both problems. It grows a large tree first and then lets data the tree was not fitted to decide which branches to keep.

R

```r
four <- data.frame(x1 = c(1, 1, 2, 2),
  x2 = c(1, 2, 1, 2), y = c(0, 10, 10, 0))
sapply(c("x1", "x2"),
  function(c) gains(four[[c]], four$y)$g)  # 0 0
```

Python

```python
four = pd.DataFrame({"x1": [1, 1, 2, 2],
    "x2": [1, 2, 1, 2], "y": [0, 10, 10, 0]})
[gains(four[c].to_numpy(), four["y"].to_numpy())[1]
 for c in ("x1", "x2")]  # [array([0.]), array([0.])]
DTR().fit(four[["x1", "x2"]], four["y"]).get_n_leaves()
# 4 leaves, each holding one case, SSE 0
```

3.1 Post-pruning, in four steps

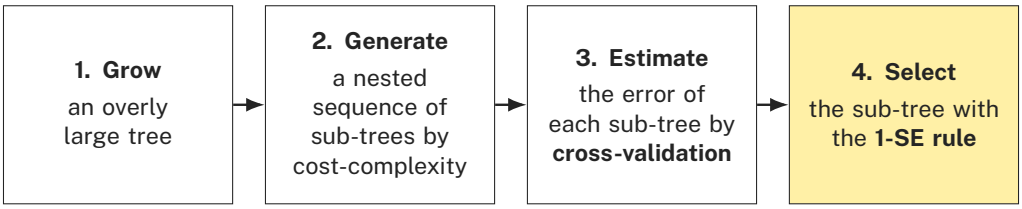


Figure 4. The four steps of post-pruning

1. Grow an overly large tree, here the nine-leaf tree.
2. Generate a sequence of smaller subtrees by charging a price for each leaf.
3. Estimate the error of each subtree by cross-validation.
4. Keep the smallest subtree whose estimate is within one standard error of the best.

3.2 A price per leaf

The rule Write $R(T)$ for the training SSE of a subtree T, and add a charge of α for every leaf,

$$R_\alpha(T) = R(T) + \alpha \times \text{leaves}.$$

$R_\alpha(T)$ is the **cost-complexity** of the subtree and α is the **complexity parameter** [1]. α is measured in units of SSE, so a leaf is worth keeping only if it lowers $R(T)$ by more than α. At $\alpha = 0$ nothing is charged and all nine leaves stay. As α rises, the cheapest tree gets smaller, and a very large α keeps only the root.

Worked example Table 3 lists the best subtree of each size with its training SSE. At $\alpha = 1$ the cost $R(T) + 1 \times$ leaves is lowest at six leaves, $16.76 + 6 = 22.76$. At $\alpha = 5$ it is lowest at five, $19.79 + 25 = 44.79$, against $16.76 + 30 = 46.76$ for six leaves and $44.56 + 20 = 64.56$ for four, because the sixth leaf saved less than 5 in SSE and the fifth saved far more.

Table 3. Training SSE and the cost at two prices of a leaf, for the best subtree of each size

Leaves	$R(T)$, training SSE	Cost at $\alpha = 1$	Cost at $\alpha = 5$
9	15.95	24.95	60.95
8	16.07	24.07	56.07
7	16.34	23.34	51.34
6	16.76	**22.76**	46.76
5	19.79	24.79	**44.79**
4	44.56	48.56	64.56

R

```r
R <- tab[, "rel error"] * fit$frame$dev[1]  # SSE
leaves <- tab[, "nsplit"] + 1
for (alpha in c(1, 5)) {      # price of one leaf
  total <- R + alpha * leaves
  cat(alpha, leaves[which.min(total)],
      round(total[1:6], 2), "\n")
}
# 1 6 270.92 162.51 98.45 48.56 24.79 22.76
# 5 5 274.92 170.51 110.45 64.56 44.79 46.76
```

Python

```python
sse = path.impurities * 60          # training SSE
leaves = np.arange(9, 0, -1)        # 9 down to 1
for alpha in (1, 5):           # price of one leaf
    total = sse + alpha * leaves
    print(alpha, leaves[total.argmin()],
          np.round(total[:6], 2))
# 1 6 [24.95 24.07 23.34 22.76 24.79 48.56]
# 5 5 [60.95 56.07 51.34 46.76 44.79 64.56]
```

3.3 The weakest link

The rule Removing a question collapses its whole branch into one leaf. That raises $R(T)$ by the branch's SSE as one leaf minus the SSE of its leaves, and it saves one leaf fewer than the branch has. The question's **price** is the rise per leaf saved,

$$\text{price} = \frac{\text{SSE as one leaf} - \text{SSE of its leaves}}{\text{leaves in the branch} - 1}.$$

Once α passes a question's price, collapsing it lowers the cost. The question with the smallest price, the **weakest link**, goes first. Its removal changes the branches above it, so their prices are worked out again, and the next weakest goes next.

Worked example The 18 cases with $x_2 \geq 6.05$, the region with mean 6.3, sit under the question $x_1 < 5.35$. As one leaf they have SSE 6.85, and in the branch's three leaves $1.46 + 1.20 + 0.75 = 3.41$, so the price is $(6.85 - 3.41)/2 = 1.72$. Inside that branch, $x_1 < 6.80$ splits 12 cases with SSE 2.37 into leaves with $1.20 + 0.75 = 1.95$, a price of 0.42, so it goes before its parent. With it gone, $x_1 < 5.35$ has two leaves, 1.46 and 2.37, and its price becomes $6.85 - 3.83 = 3.02$. Across the whole grown tree the smallest price belongs to $x_2 < 4.85$, 0.115, so it is the weakest link. The listing prints every question's price before anything is removed, and Figure 3 labels each question with the price at which it finally goes.

R

```r
fr <- fit$frame; id <- as.numeric(rownames(fr))
below <- function(i) {   # leaves and SSE in i's branch
  k <- which(id == i)
  if (fr$var[k] == "<leaf>") return(c(1, fr$dev[k]))
  below(2 * i) + below(2 * i + 1)
}
for (i in id[fr$var != "<leaf>"]) {    # each question
  b <- below(i); k <- which(id == i)
  cat(i, fr$n[k], round(fr$dev[k], 2), b[1],
      round((fr$dev[k] - b[2]) / (b[1] - 1), 3), "\n")
}
# 1 60 269.92 9 31.746     node, n, SSE, leaves, price
# 2 12 4.39 2 0.276
# 3 48 156.12 7 24.047
# 7 37 87.45 6 15.845
# 14 19 29.7 3 12.444
# 29 13 2.52 2 0.115       the weakest link
# 15 18 6.85 3 1.722
# 30 12 2.37 2 0.422
```

Python

```python
t = base.tree_                       # the grown tree
L, R = t.children_left, t.children_right
node = t.impurity * t.n_node_samples  # each SSE
def below(i):     # leaves and SSE in node i's branch
    if L[i] < 0:
        return 1, node[i]
    (k1, s1), (k2, s2) = below(L[i]), below(R[i])
    return k1 + k2, s1 + s2
for i in np.flatnonzero(L >= 0):    # each question
    k, s = below(i)
    print(i, t.n_node_samples[i], round(node[i], 2),
          k, round((node[i] - s) / (k - 1), 3))
# 0 60 269.92 9 31.746     node, n, SSE, leaves, price
# 1 12 4.39 2 0.276
# 4 48 156.12 7 24.047
# 6 37 87.45 6 15.845
# 7 19 29.7 3 12.444
# 8 13 2.52 2 0.115        the weakest link
# 12 18 6.85 3 1.722
# 14 12 2.37 2 0.422
```

The two languages number nodes differently. R numbers the children of node i as $2i$ and $2i+1$, so its weakest link is node 29. scikit-learn numbers nodes in the order it grows them, so the same question is node 8. Both print the same sizes, SSEs and prices.

3.4 The pruning sequence

The rule Raise α slowly from 0 and the questions go in the order of their prices. Each subtree is the one before it with one more branch removed, so the subtrees are **nested**, and a branch once removed never comes back. The search over every possible subtree becomes a list with one subtree per size, and each member of the list has the lowest training SSE of any subtree of its size.

Worked example Table 4 is the sequence for the 60 cases. Each removal here takes out one split, so the α column is the rise in SSE that removal costs. The first three removals cost 0.115, 0.276 and 0.422 in SSE, almost nothing, because they undo splits inside a single region. Going from six leaves to five costs 3.02. Going from five to four costs 24.77, eight times more, because that split separates two real regions. Read each row as *a smaller tree for a small rise in training error*, and the table shows where the rise stops being small.

Table 4. The pruning sequence, with the training SSE, the price α at which each subtree takes over, and its ten-fold cross-validated MSE with standard error

Leaves	$R(T)$, SSE	α, its price	10-fold MSE	SE
9	15.95	0.000	0.753	0.381
8	16.07	0.115	0.753	0.381
7	16.34	0.276	0.753	0.381
6	16.76	0.422	0.753	0.381
5	**19.79**	**3.022**	**0.803**	0.387
4	44.56	24.774	1.209	0.528
3	95.45	50.891	2.063	0.705
2	160.51	65.061	2.955	0.668
1	269.92	109.404	4.396	0.734

Python

```
pd.DataFrame({"leaves": [DTR(ccp_alpha=c, **kw)
    .fit(X, y).get_n_leaves() for c in a],
    "alpha_x_60": (a * 60).round(3),
    "train_sse": (path.impurities * 60).round(2),
    "cv_mse": e.round(3), "se": se.round(3)})
#    leaves  alpha_x_60  train_sse  cv_mse     se
# 0       9       0.000      15.95   0.753  0.381
# 1       8       0.115      16.07   0.753  0.381
# 2       7       0.276      16.34   0.753  0.381
# 3       6       0.422      16.76   0.753  0.381
# 4       5       3.022      19.79   0.803  0.387
# 5       4      24.774      44.56   1.209  0.528
# 6       3      50.891      95.45   2.063  0.705
# 7       2      65.061     160.51   2.955  0.668
# 8       1     109.404     269.92   4.396  0.734
round(e.min() + se[e.argmin()], 3)  # 1.134, 1-SE line
```

3.5 Choosing the size by cross-validation

What it is for The sequence offers nine candidates. Training error always prefers the largest, so the choice needs an estimate of each candidate's error on cases it was not fitted to, the same role cross-validation played in choosing K for kNN.

The estimate Split the 60 cases into 10 folds of 6, here the ten consecutive blocks of six rows. For each subtree in the sequence, regrow the tree ten times, each time on 54 cases, prune it at that subtree's α, and score it on the 6 left out. The mean of the ten fold errors is its cross-validated MSE. The standard deviation s of the ten fold errors, divided by $\sqrt{10}$, is its **standard error**, the usual standard error of a mean of fold scores, which says how much the mean would move with a different split into folds.

Worked example For the five-leaf row the ten fold errors are 0.623, 4.236, 0.225, 0.260, 0.351, 0.573, 0.217, 0.788, 0.244 and 0.511. They add to 8.028, so the cross-validated MSE is 0.803. Their standard deviation is $s = 1.222$, and the standard error is $1.222/\sqrt{10} = 0.387$. One fold dominates. Fold 2 holds the cases $(4.8, 3.1)$ and $(3.8, 3.5)$, which sit just above the low region. Without them the fold's tree puts its first cut at $x_2 < 3.65$ and predicts 1.90 for both, squared errors 8.91 and 13.73. The other nine scores lie between 0.217 and 0.788, so most of the standard error comes from that one fold.

R

```
fold_mse <- function(grow) sapply(1:10, function(k) {
  m <- grow(d[fold != k, ])          # fit on 54 cases
  mean((predict(m, d[fold == k, ]) - d$y[fold == k])^2)
})
tree5 <- function(s) prune(rpart(y ~ x1 + x2, s,
  control = rpart.control(cp = 0, minbucket = 6,
    minsplit = 12, xval = 0)), cp = tab[row, "CP"])
f <- fold_mse(tree5)                 # the 5-leaf row
round(f, 3)   # 0.623 4.236 0.225 0.260 0.351 0.573 ...
round(c(mean(f), sd(f), sd(f) / sqrt(10)), 3)
# 0.803 1.222 0.387, the mean, sd and SE
tree5(d[fold != 2, ])$splits[1, "index"]   # 3.65
```

Python

```
s = -cross_val_score(DTR(ccp_alpha=a[4], **kw),
    X, y, cv=f10,                    # the 5-leaf row
    scoring="neg_mean_squared_error")
s.round(3)  # [0.623 4.236 0.225 0.26 0.351 0.573 ...]
round(s.mean(), 3), round(s.std(ddof=1), 3), round(
    s.std(ddof=1) / np.sqrt(10), 3)
# (0.803, 1.222, 0.387), the mean, sd and SE
out = X.index[6:12]                  # fold 2's six cases
m2 = DTR(ccp_alpha=a[4], **kw).fit(X.drop(out),
                                   y.drop(out))
m2.tree_.threshold[0], m2.predict(X.loc[[9, 11]])
# 3.65 [1.9 1.9], (4.8, 3.1) and (3.8, 3.5)
```

The 1-SE rule The **one-standard-error rule**, 1-SE for short, takes the smallest cross-validated MSE, adds its standard error, and keeps the *smallest* subtree at or below that line [1]. A difference smaller

than one standard error is inside the noise of the estimate, so it does not justify a leaf. The curve of error against size usually falls fast, then runs nearly flat, and which tree on the flat stretch scores lowest is decided by the folds that happened to be drawn [4]. The same drift made the depth search on `Glass` pick the deepest tree on a flat curve.

Worked example The last two columns of Table 4 are the estimates. The smallest MSE, 0.753, is shared by the trees with six to nine leaves, with standard error 0.381. The line is $0.753 + 0.381 = 1.134$. The five-leaf tree scores 0.803, under the line, and the four-leaf tree scores 1.209, over it, so the rule keeps **five leaves** (Figure 5).

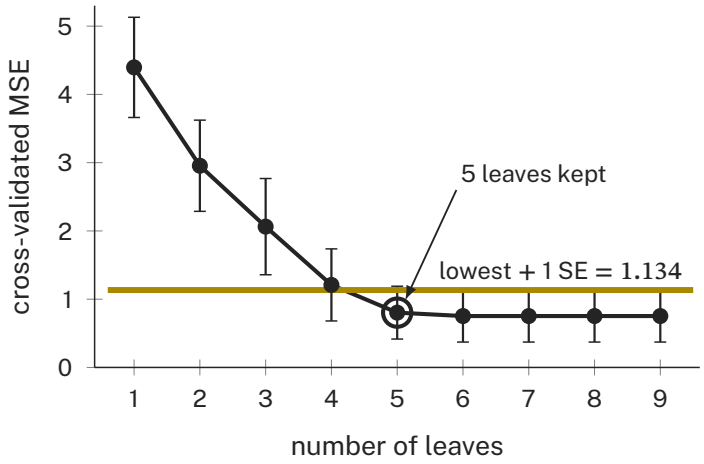


Figure 5. Ten-fold cross-validated MSE of each subtree, with one standard error as bars and the 1-SE line

The x-SE rule One standard error is a convention. The **x-SE rule** draws the line at the smallest MSE plus x standard errors. At $x = 0$ it keeps the tree with the smallest estimate, and a larger x trades estimated error for a smaller tree. In Python $x = 0$ keeps six leaves, $x = 0.5$ and $x = 1$ keep five, and $x = 2$, with the line at $0.753 + 2 \times 0.381 = 1.516$, keeps four. R, whose estimates are computed differently (Section 4.1), keeps seven, six, five and four. The two agree at the usual $x = 1$.

R

```r
for (x in c(0, 0.5, 1, 2)) {        # the x-SE rule
  lim <- tab[best, "xerror"] + x * tab[best, "xstd"]
  cat(x, tab[min(which(tab[, "xerror"] <= lim)),
    "nsplit"] + 1, "\n")
}
# 0 7, 0.5 6, 1 5, 2 4: x and the leaves kept
```

Python

```python
for x in (0, 0.5, 1, 2):            # the x-SE rule
    keep = e <= e.min() + x * se[e.argmin()]
    print(x, DTR(ccp_alpha=a[keep].max(), **kw)
          .fit(X, y).get_n_leaves())
# 0 6, 0.5 5, 1 5, 2 4: x and the leaves kept
```

3.6 The result

Figure 6 is the pruned tree. Its training SSE is 19.79, 24% above the nine-leaf tree's 15.95, as it must be, since pruning cannot improve the fit to the training cases. Its five leaves hold 12, 11, 13, 6 and 18 cases, and the regions block of the listing checks that every case in a leaf comes from one true region, with leaf means 1.08, 2.32, 4.72, 2.26 and 6.29 against true means 0.75, 2.2, 4.9, 2.3 and 6.3. The four removed splits were dividing noise. The leaf of the worked example is still there, node 28 in R and node 7 in scikit-learn, with SSE 2.41 and Err 0.402.

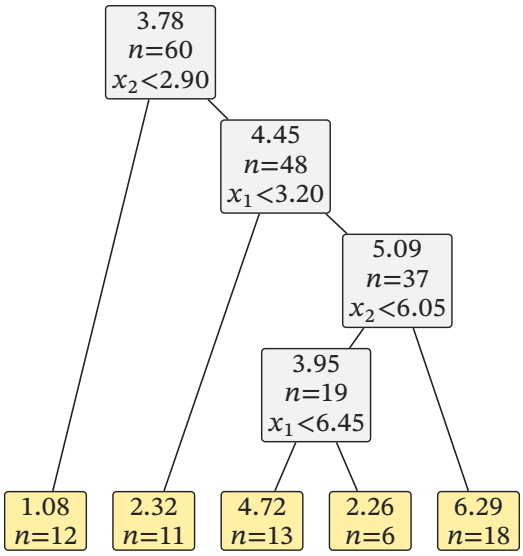


Figure 6. The pruned five-leaf tree

R

```
print(pruned, digits = 3)
#  1) root 60 270.00 3.78
#    2) x2< 2.9 12 4.39 1.08 *
#    3) x2>=2.9 48 156.00 4.45
#      6) x1< 3.2 11 3.61 2.32 *
#      7) x1>=3.2 37 87.40 5.09
#       14) x2< 6.05 19 29.70 3.95
#         28) x1>=6.45 6 2.41 2.26 *
#         29) x1< 6.45 13 2.52 4.72 *
#       15) x2>=6.05 18 6.85 6.29 *
```

Python

```
print(export_text(pruned, feature_names=["x1", "x2"]))
# |--- x2 <= 2.90
# |   |--- value: [1.08]
# |--- x2 >  2.90
# |   |--- x1 <= 3.20
# |   |   |--- value: [2.32]
# |   |--- x1 >  3.20
# |   |   |--- x2 <= 6.05
# |   |   |   |--- x1 <= 6.45
# |   |   |   |   |--- value: [4.72]
# |   |   |   |--- x1 >  6.45
# |   |   |   |   |--- value: [2.26]
# |   |   |--- x2 >  6.05
# |   |   |   |--- value: [6.29]
p = pruned.tree_                    # node 7 is the leaf
p.n_node_samples[7], p.impurity[7].round(3)  # 6 0.402
```

Python

```
truth = [region(a, b) for a, b in zip(
    X["x1"], X["x2"], strict=True)]
d.assign(leaf=pruned.apply(X), truth=truth).groupby(
    "leaf").agg(n=("y", "size"), mean=("y", "mean"),
    regions=("truth", "nunique"),
    truth=("truth", "first")).round(2)
#       n  mean  regions  truth
# leaf
# 1    12  1.08        1   0.75
# 3    11  2.32        1   2.20
# 6    13  4.72        1   4.90
# 7     6  2.26        1   2.30
# 8    18  6.29        1   6.30
```

4 Pruning in R and in Python

What it is for Both libraries implement the four steps, but they divide the work differently.

R

```r
library(rpart); library(rpart.plot)
fold <- rep(1:10, each = 6)          # 10 blocks of 6
fit <- rpart(y ~ x1 + x2, data = d,
  method = "anova",   # cp = 0 grows large
  control = rpart.control(cp = 0, minbucket = 6,
    minsplit = 12, xval = fold))
printcp(fit)   # CP nsplit rel error xerror xstd
tab <- fit$cptable                   # 1-SE rule
best <- which.min(tab[, "xerror"])
lim <- tab[best, "xerror"] + tab[best, "xstd"]
row <- min(which(tab[, "xerror"] <= lim))
pruned <- prune(fit, cp = tab[row, "CP"])
rpart.plot(pruned)                   # 5 leaves
```

Python

```python
kw = {"min_samples_leaf": 6, "random_state": 3141}
base = DTR(**kw).fit(X, y)           # grow large
path = base.cost_complexity_pruning_path(X, y)
a = path.ccp_alphas                  # one per subtree
f10 = KFold(10)                      # 10 blocks of 6
e, se = [], []                       # CV error, its SE
for c in a:
    s = -cross_val_score(DTR(ccp_alpha=c, **kw),
        X, y, cv=f10,
        scoring="neg_mean_squared_error")
    e.append(s.mean())
    se.append(s.std(ddof=1) / np.sqrt(10))
e, se = np.array(e), np.array(se)
ok = e <= e.min() + se[e.argmin()]  # 1-SE rule
pruned = DTR(ccp_alpha=a[ok].max(), **kw).fit(X, y)
```

`rpart` [4] runs its own cross-validation while it fits, so `printcp()` already holds the estimate for every subtree, and the listing applies the 1-SE rule to `fit$cptable` and passes the chosen row's `CP` to `prune()`. Its `xval` setting takes either a number of folds, drawn at random, or a vector with one fold number per case. The listing passes `fold`, the ten blocks of six, so R and Python score the same folds. scikit-learn's `cost_complexity_pruning_path()` [3] returns the α sequence and nothing else, so the loop in the middle of the Python listing is steps 3 and 4 written out with `cross_val_score` on ten folds of six. Both grow the same nine-leaf tree and both keep five leaves.

4.1 Reading the cp table

Table 5 is what `printcp()` prints, rounded to four decimals. Every column is divided by the root's SSE, 269.92, or equivalently by its Err, 4.4986, so the numbers are unitless.

- `nsplit` is the number of splits, so the tree has `nsplit`+1 leaves.
- `rel error` is $R(T)$ over the root's. The five-leaf row has $19.79/269.92 = 0.0733$. It only falls.
- `CP` is α divided by 269.92. The five-leaf row's 0.01120 is $3.022/269.92$, the price at which that tree takes over, and the root row's 0.40533 is $109.404/269.92$, the drop the root question buys.
- `xerror` is `rpart`'s cross-validated error on the same scale, so multiplying by 4.4986 gives an MSE. The bottom three rows give $0.1674 \times 4.4986 = 0.753$, the same lowest MSE the Python folds give.
- `xstd` is the standard error of `xerror`. `rpart` computes it as the standard error of the mean of the 60 per-case squared errors, rather than from the ten fold means.

Table 5. The cp table `printcp()` prints for the 60 cases

CP	nsplit	rel error	xerror	xstd
0.40533	0	1.0000	1.0320	0.1093
0.24104	1	0.5947	0.7153	0.1031
0.18854	2	0.3536	0.5021	0.1070
0.09178	3	0.1651	0.2766	0.0821
0.01120	4	0.0733	0.1992	0.0766
0.00156	5	0.0621	0.1737	0.0600
0.00102	6	0.0605	0.1674	0.0600
0.00043	7	0.0595	0.1674	0.0600
0.00000	8	0.0591	0.1674	0.0600

The smallest `xerror` is 0.1674, at `nsplit` 6, with `xstd` 0.0600. The 1-SE line is $0.1674 + 0.0600 = 0.2274$, and the first row at or below it is `nsplit` 4, five leaves, the same choice the Python folds make. On the same folds the other rows still differ from Python's, for two reasons beyond the standard error. `rpart` scores each row at a typical α inside the row's range, the geometric mean of its `CP` and the one above, rather than at the `CP` itself [4], and the five-leaf row's `xerror` gives $0.1992 \times 4.4986 = 0.896$ rather than Python's 0.803. scikit-learn's `ccp_alpha` is α divided by the 60 cases, because its node error is a mean rather than a sum, so the listing multiplies by 60 to print SSE units.

4.2 The defaults are not the same model

Table 6 lists where the two libraries' defaults diverge. `rpart` was written to hand back a model a person can read, so it stops early without being asked. Its `cp = 0.01` refuses any split that lowers the SSE by less than 1% of the root's, and its smallest leaf of 7 cases is what gives four leaves here. scikit-learn returns the unregularized fit, 60 leaves, depth 12, training SSE 0, and leaves the penalty to you. Report the data, the error estimate and the control settings. "We fitted a decision tree" does not identify a model.

Table 6. Default pruning and stopping rules in R and Python

	R · `rpart`	Python · `DecisionTree*`
Pruning by default	**Yes.** With `cp = 0.01` a split is attempted only if it lowers the training SSE by at least 1% of the root's, so the tree is pruned *as it grows*.	**No.** With `ccp_alpha = 0.0` the tree grows to purity, and it is pruned only if you pass a non-zero `ccp_alpha`.
Other stopping rules	`minsplit = 20`, `minbucket = round(minsplit/3) = 7`, `maxdepth = 30`.	`min_samples_split = 2`, `min_samples_leaf = 1`, `max_depth = None`.
Error estimate	Built in. `printcp()` and `plotcp()` report `xerror` and `xstd` for every subtree.	None. `cost_complexity_pruning_path()` returns the α sequence only, and you estimate the error by cross-validation.
Split criterion	Gini for `method = "class"`, least squares for `method = "anova"`.	`criterion="gini"` for the classifier, `"squared_error"` for the regressor.
On these 60 cases	Defaults give a **4-leaf** tree. A leaf needs 7 cases, and one region holds only 6.	Defaults give a **60-leaf** tree, depth 12, training SSE $= 0$.

4.3 Pruning a classification tree

A classification tree is pruned by the same four steps. Only $R(T)$ changes. `rpart` with `method = "class"` counts the training cases a subtree misclassifies, so its `rel error` and `xerror` are error rates over the root's, and the root's is the share of cases outside the majority class. scikit-learn's `DecisionTreeClassifier` uses the Gini impurity of the leaves, each weighted by its share of the cases [3]. Either way the sequence is nested and cross-validation picks from it.

Worked example `Glass`, the six-class data whose tree depth was tuned before, on the same five folds. Grown to purity, the tree has 50 leaves in both languages. Python scores every subtree by balanced accuracy, a measure to maximize, so the 1-SE rule keeps the smallest subtree whose score is at least the best minus one standard error. It keeps 8 leaves, with a cross-validated balanced accuracy

of 0.665 against 0.597 for the 50-leaf tree. R reads its cp table as before. Its 1-SE line is 0.4749, and the first row under it, at 0.4710, also has 8 leaves.

R

```r
data(Glass, package = "mlbench")
set.seed(3141)                          # 5 frozen folds
gf <- caret::createFolds(Glass$Type, k = 5,
                         list = FALSE)
g <- rpart(Type ~ ., Glass, method = "class",
  control = rpart.control(cp = 0, minsplit = 2,
    minbucket = 1, xval = gf))      # grow to purity
gt <- g$cptable; b <- which.min(gt[, "xerror"])
r <- min(which(gt[, "xerror"] <=
  gt[b, "xerror"] + gt[b, "xstd"]))
round(c(line = gt[b, "xerror"] + gt[b, "xstd"],
  kept = gt[r, "xerror"]), 4)       # 0.4749 0.4710
sapply(list(g, prune(g, cp = gt[r, "CP"])),
  function(m) sum(m$frame$var == "<leaf>"))  # 50 8
```

Python

```python
url = ("https://archive.ics.uci.edu/ml/"
       "machine-learning-databases/glass/glass.data")
glass = pd.read_csv(url, header=None, index_col=0)
Xg, yg = glass.iloc[:, :-1], glass.iloc[:, -1]
f5 = SKF(5, shuffle=True, random_state=3141)
ag = DTC(random_state=0).fit(Xg, yg) \
    .cost_complexity_pruning_path(Xg, yg).ccp_alphas
sc = np.array([cross_val_score(DTC(ccp_alpha=c,
    random_state=0), Xg, yg, cv=f5,
    scoring="balanced_accuracy") for c in ag])
m, s = sc.mean(1), sc.std(1, ddof=1) / np.sqrt(5)
k = np.flatnonzero(m >= m.max() - s[m.argmax()]).max()
[DTC(ccp_alpha=c, random_state=0).fit(Xg, yg)
 .get_n_leaves() for c in (0, ag[k])]   # [50, 8]
m[0].round(3), m[k].round(3)            # 0.597 0.665
```

The two libraries score different things here, the error rate in R and balanced accuracy in Python, and they happen to agree on the size. To prune by balanced accuracy in R, loop over the `CP` column with your own folds, as the Python listing loops over α. To report the pruned tree's accuracy honestly, run all four steps inside each outer fold, as the depth was tuned inside.

5 When to use a regression tree

On these data the pruned tree's ten-fold MSE is 0.80 against 2.67 for linear regression on the same folds, because the target jumps between regions and a single plane cannot follow it. The comparison would turn the other way if y rose smoothly with x_1 and x_2, since the tree could only approximate a slope with many steps, each fitted on few cases.

R

```r
lin <- function(s) lm(y ~ x1 + x2, s)
round(mean(fold_mse(tree5)), 2)     # 0.8
round(mean(fold_mse(lin)), 2)       # 2.67
round(predict(lin(d), new), 2)      # 4.88 7.33
```

Python

```python
for m in (pruned, LinearRegression()):
    s = -cross_val_score(m, X, y, cv=f10,
        scoring="neg_mean_squared_error")
    print(type(m).__name__, round(s.mean(), 2))
# DecisionTreeRegressor 0.8
# LinearRegression 2.67
line = LinearRegression().fit(X, y)
line.predict(new).round(2)    # [4.88 7.33]
```

The 0.80 is flattering. The same ten folds chose α, so the tree was partly tuned on the cases that score it, the drift the nested loop removed when the depth was tuned. Running all four steps inside each outer training set, with nine inner folds of six, gives 1.15 in Python and 1.17 in R. Inside a fold the rule keeps four or five leaves. The honest estimate is well above 0.80 and still far below the line's 2.67.

R

```r
inside <- function(s) {              # steps 1 to 4
  f <- rpart(y ~ x1 + x2, s, control = rpart.control(
    cp = 0, minbucket = 6, minsplit = 12,
    xval = rep(1:9, each = 6)))      # 9 blocks of 6
  t <- f$cptable; b <- which.min(t[, "xerror"])
  r <- min(which(t[, "xerror"] <=
    t[b, "xerror"] + t[b, "xstd"]))
  prune(f, cp = t[r, "CP"])
}
round(mean(fold_mse(inside)), 2)  # 1.17, alpha inside
```

Python

```python
def prune_1se(Xt, yt):           # steps 1 to 4
    al = DTR(**kw).fit(Xt, yt) \
        .cost_complexity_pruning_path(Xt, yt).ccp_alphas
    sc = np.array([-cross_val_score(
        DTR(ccp_alpha=c, **kw), Xt, yt, cv=KFold(9),
        scoring="neg_mean_squared_error") for c in al])
    m, s = sc.mean(1), sc.std(1, ddof=1) / np.sqrt(9)
    keep = m <= m.min() + s[m.argmin()]
    return DTR(ccp_alpha=al[keep].max(), **kw).fit(
        Xt, yt)
inner = [np.mean((prune_1se(X.iloc[tr], y.iloc[tr])
    .predict(X.iloc[te]) - y.iloc[te]) ** 2)
    for tr, te in f10.split(X)]
round(np.mean(inner), 2)  # 1.15, alpha chosen inside
```

- **Use it** when the target changes in jumps, when predictors interact (the effect of x_1 here depends on whether x_2 is above 6.1), or when the model must be read as a handful of rules.
- **It costs** a step function, a prediction that stops at the last leaf's mean outside the training range, and a setting, α, that must be chosen by cross-validation. Its structure moves with the sample when two questions nearly tie, as the weather tree's root did. Here the root question removes 1.82 against 0.97 for the best rival, a wide margin.
- **It cannot** extrapolate a trend, or give a smooth response, however many leaves it has.

6 Common misconceptions

Common misconceptions

- *"Pruning improves the fit."* It does the opposite, by construction. The training SSE rose from 15.95 to 19.79. The aim is better predictions on new cases, and cross-validation checks whether that happens.
- *"The complexity parameter is a tuning knob you set to taste."* It is a price per leaf. Each α picks one member of the nested sequence, and cross-validation chooses which.
- *"cp and ccp_alpha are the same number."* They differ from α by the root's SSE, 269.92, and by $n = 60$. Passing an `rpart` `cp` straight to `ccp_alpha` gives a tree of the wrong size.
- *"The 1-SE rule gives up accuracy."* On the estimate it gives up at most one standard error, which is inside the noise. Here it gave up 0.05 in MSE and four leaves that were splitting noise.
- *"The subtree with the lowest cross-validated error is the best tree."* On the flat stretch the lowest score is decided by the folds. On the same ten folds Python's lowest is shared by six to nine leaves and R's falls at seven, while both keep five under the 1-SE rule.

7 Summary

1. A regression tree is grown by the same recursive partitioning as a classification tree. A leaf predicts the mean of its cases, the number with the smallest mean squared error, and a question is worth the drop from the node's error to the pooled error of its sides, or equally the parent's SSE minus its sides' SSE.

2. Its predictions are steps, constant in each leaf's rectangle, and never outside the range of the training targets.
3. Charging α per leaf, the cost-complexity $R(T) + \alpha \times$ leaves, turns the search over all subtrees into a nested sequence. Each question has a price, its branch's saving per extra leaf, and the cheapest goes first.
4. Ten-fold cross-validation scores every subtree, with a standard error from the fold scores, and the 1-SE rule keeps the smallest within one standard error of the best. On the 60 cases it keeps five leaves, one per true region.
5. `rpart` prunes by default and estimates the error for you. scikit-learn grows to purity and does neither unless you ask. A classification tree is pruned the same way. Report the settings, and choose α inside the folds that score the model.

References

[1] L. Breiman, J. Friedman, R. Olshen, and C. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[2] S. Milborrow. *Plotting rpart Trees with the rpart.plot Package*. Vignette of the `rpart.plot` package.

[3] scikit-learn developers. *Decision Trees*. User guide, section 1.10, including Minimal Cost-Complexity Pruning. URL: `https://scikit-learn.org/stable/modules/tree.html`.

[4] T. Therneau and E. Atkinson. *An Introduction to Recursive Partitioning Using the RPART Routines*. Tech. rep. Distributed as the `rpart` package vignette. Mayo Foundation.

[5] L. Torgo. "Inductive learning of tree-based regression models". PhD thesis. Department of Computer Science, Faculty of Sciences, University of Porto, 1999.

[6] L. Torgo. "Regression Trees". In: *Encyclopedia of Machine Learning*. Ed. by C. Sammut and G. I. Webb. Springer, 2011, pp. 842–845.